

Timers

Unblocking the CPU

The Software Delay Problem

- ▶ **Blocking Code:** Functions like `sleep()` or `delay()` are entirely blocking. The CPU is trapped running an empty loop checking the time.
- ▶ **Zero Multitasking:** While blocked, the microcontroller cannot read sensor data, process inputs, or respond to emergencies.

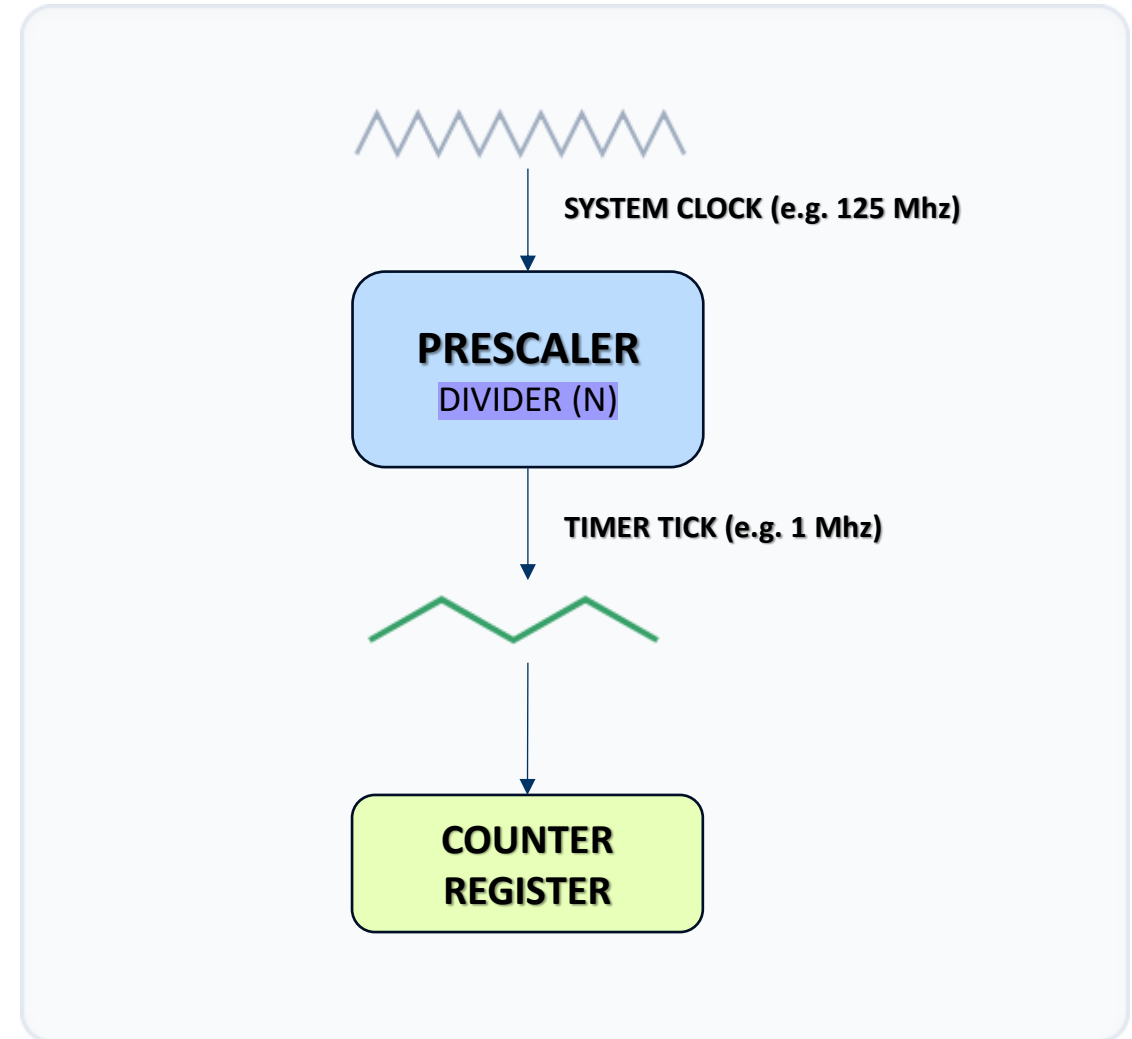


The Gear Reducer: The Prescaler

Timers require a clock signal to increment. However, system clocks are extremely fast (e.g., 125 MHz). A 16-bit timer counting at 125 MHz overflows in a fraction of a millisecond.

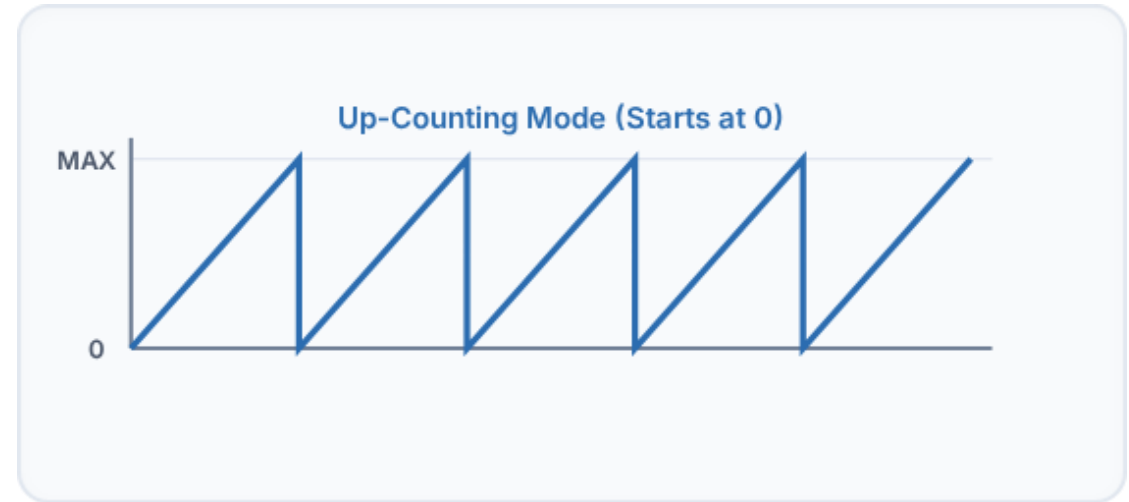
The Prescaler mathematically divides the high-frequency system clock before it reaches the timer.

If you set a Prescaler of 125, the actual Timer Register only counts up once for every 125 system clock ticks, slowing it down to a usable 1 MHz (1 tick = 1 microsecond).

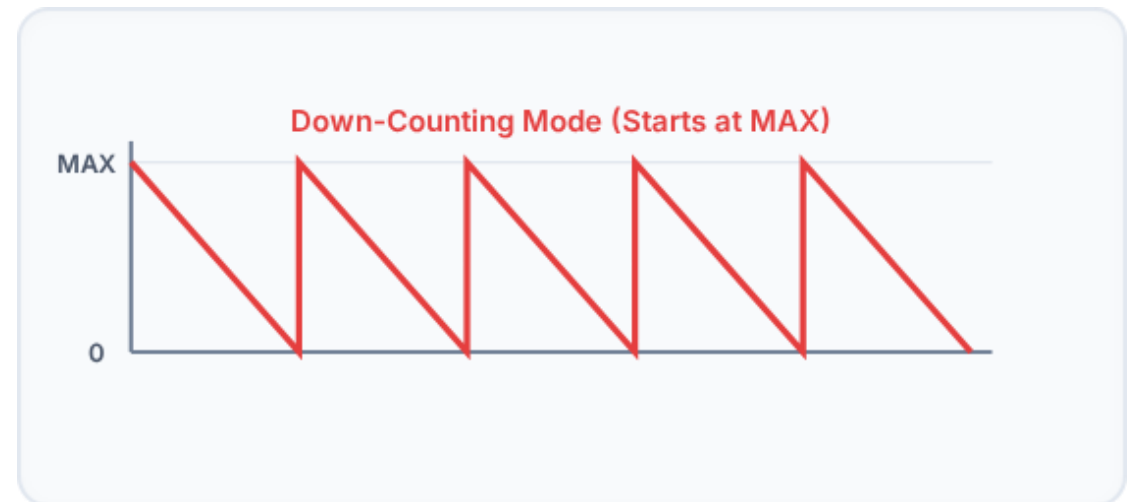


Timer Config: Up & Down Counting

- ▶ **Up-Counting (Sawtooth):** The standard mode.
The hardware starts at 0, increments to a maximum value, and instantly resets back to 0.
Analogy: A stopwatch counting elapsed time up from 0:00.

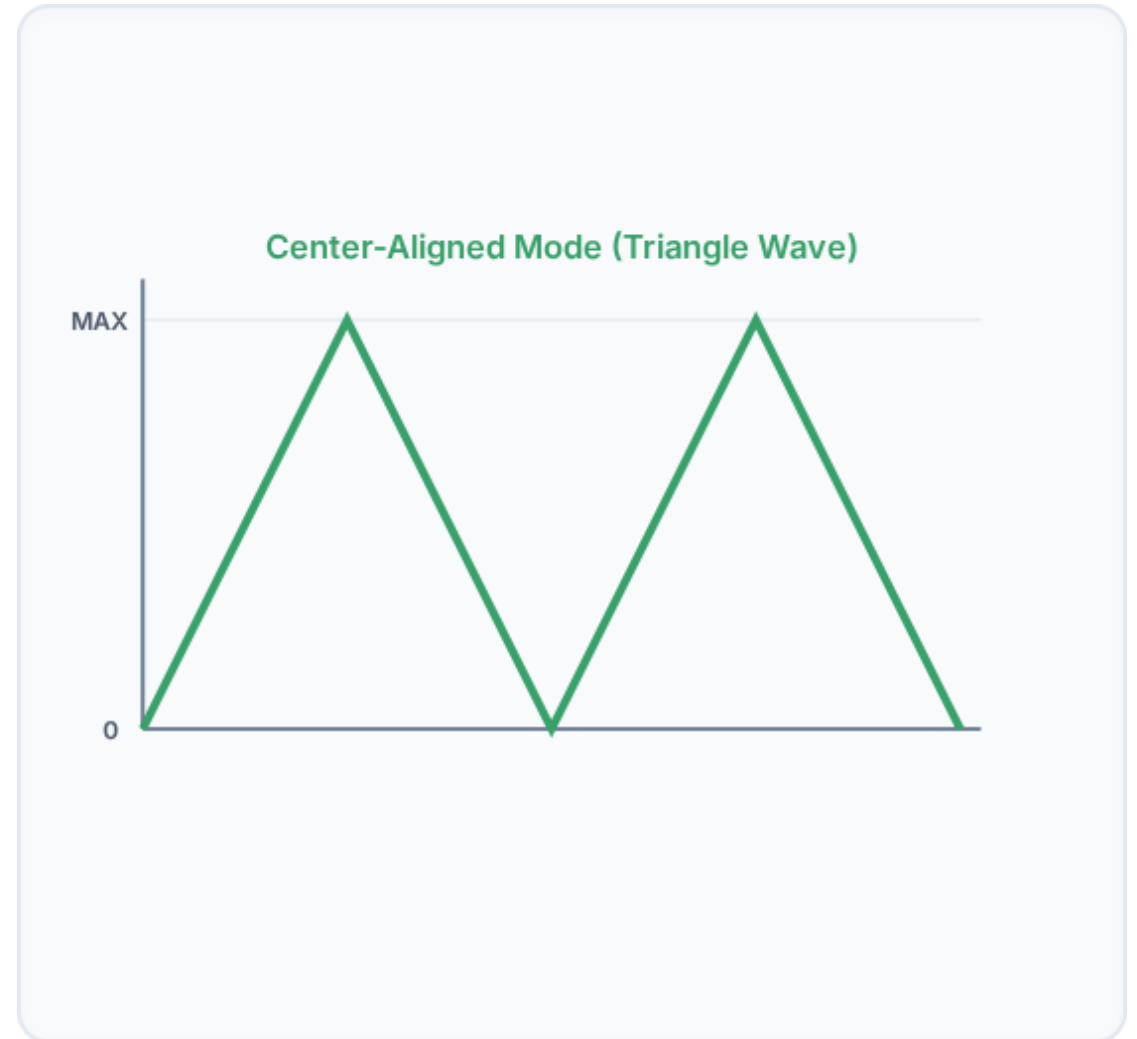


- ▶ **Down-Counting (Reverse Sawtooth):** The reverse.
You load a maximum value, and the hardware decrements to 0 before automatically reloading the maximum.
Analogy: A microwave timer counting down to 0.



Timer Config: Center-Aligned (Up/Down)

- ▶ **Up/Down Counting (Center-Aligned):** Instead of snapping back to zero, the timer counts up to MAX, turns around, and gracefully counts back down to zero.
- ▶ **The Triangle Wave:** This creates a perfectly symmetrical triangle wave in hardware.
- ▶ **Why use this?** This is the secret weapon for **Motor Control** and generating **symmetrical PWM** (Pulse Width Modulation). It aligns the pulses perfectly in the center, which drastically reduces electromagnetic noise and makes motors run smoother.



When do we get interrupted?

The hardware timer signals the CPU via an **Interrupt Service Routine (ISR)**. But exactly *when* does it pull the alarm?

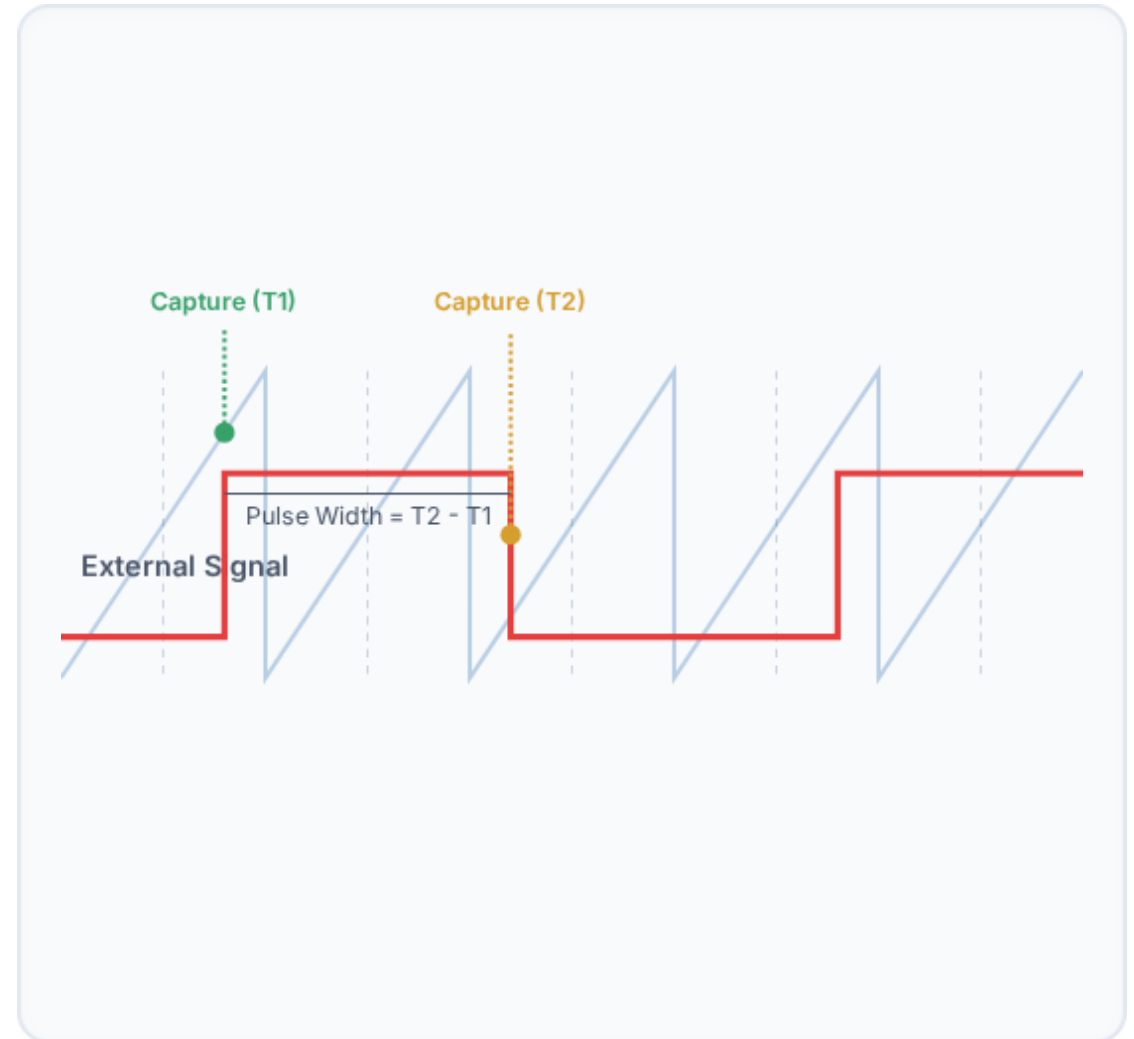
1. Overflow Interrupt: Fires when the counter hits its absolute limit and rolls over. (The peak of the sawtooth).

2. Compare Match Interrupt: You load a specific number into a "Compare Register". As the counter runs, hardware constantly compares them. When they match, it fires an interrupt. This allows triggering events at exact intervals *before* an overflow happens.



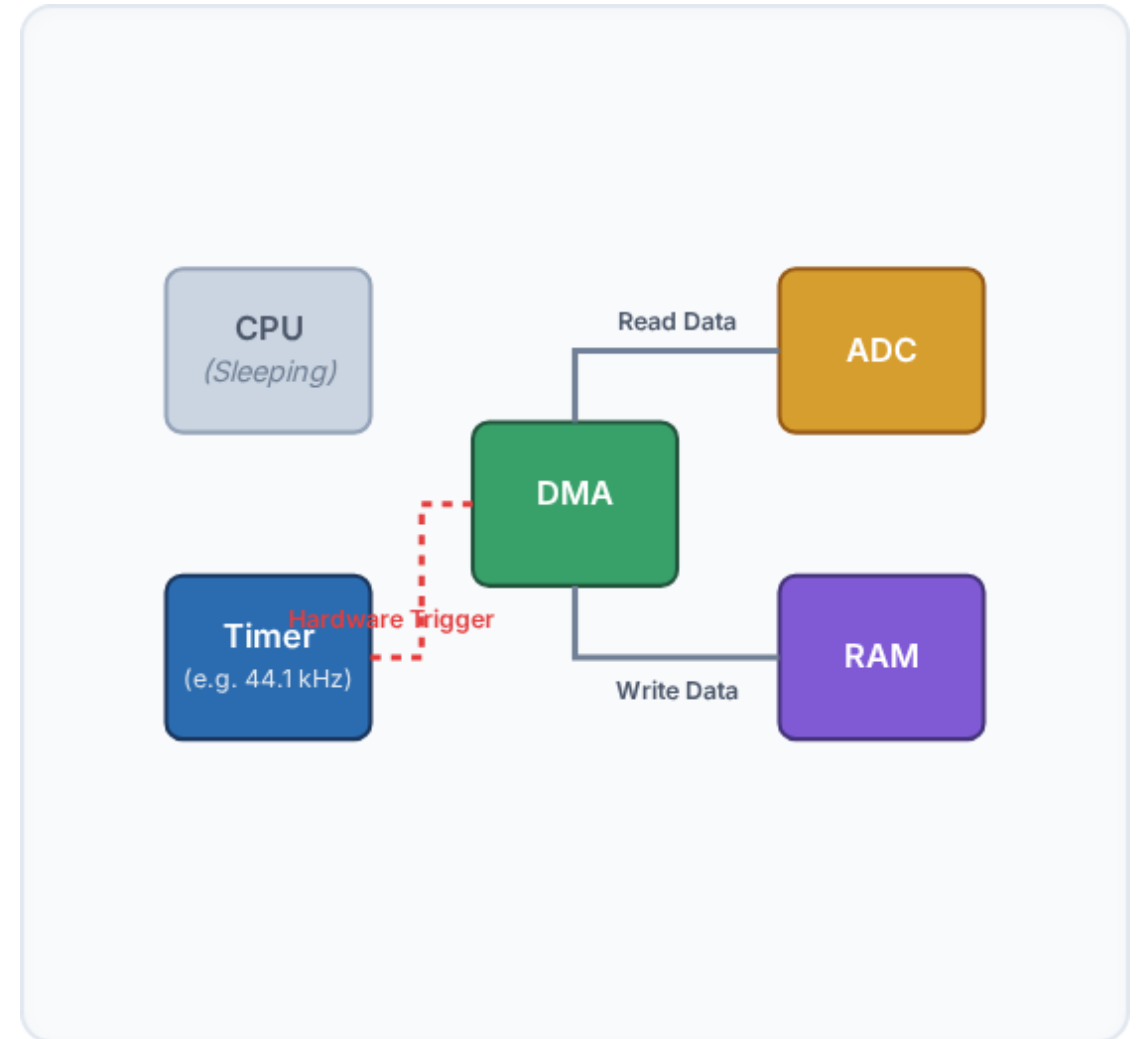
Input Capture (The Digital Stopwatch)

- ▶ While Output Compare *creates* events, Input Capture *measures* external events with microscopic precision.
- ▶ **How it works:** A GPIO pin is routed directly to a timer. When the pin changes state (e.g., goes HIGH), the hardware instantly copies the timer's current count into a special "Capture Register" and fires an interrupt.
- ▶ **The Analogy:** A runner breaking a laser beam at a finish line. The laser automatically records the exact time on a clipboard before the referee (CPU) reacts.
- ▶ **Use Cases:** Measuring motor RPM (Hall effect sensors), decoding IR remote signals, reading RC receiver pulse widths.



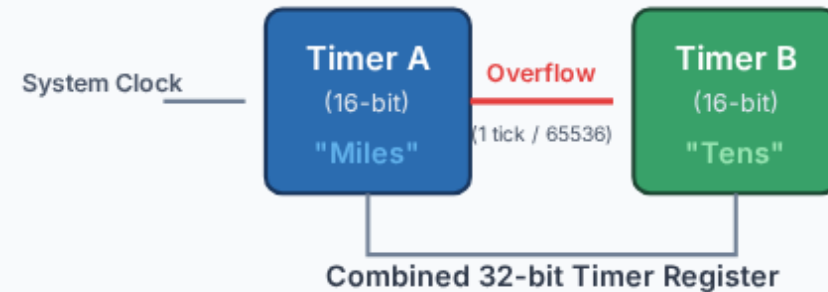
Timer-Triggered DMA (The Invisible Hand)

- ▶ Direct Memory Access (DMA) allows data to move around the chip without the CPU lifting a finger. This is the ultimate evolution of non-blocking code.
- ▶ **How it works:** Configure a timer to overflow at a specific rate (e.g., 44.1 kHz). Instead of firing a CPU interrupt, the overflow sends a hardware signal directly to the DMA controller.
- ▶ The DMA grabs one sample from the Analog-to-Digital Converter (ADC) and puts it into RAM, entirely independently.
- ▶ **Use Cases:** High-quality audio sampling, reading continuous sensor streams, driving NeoPixels (WS2812B) with zero CPU overhead.



Timer Cascading (The Odometer)

- ▶ Sometimes, a 16-bit timer (which only counts to 65,535) isn't large enough, even with a prescaler, to track time over days or weeks.
- ▶ **How it works:** Hardware timers can be chained together. Timer A counts the system clock. When **Timer A overflows**, its **output is routed** as the clock input to Timer B.
- ▶ **The Analogy:** An analog car odometer. When the "miles" wheel completes a full rotation, it physically clicks the "tens of miles" wheel forward by one notch.
- ▶ This allows creating 32-bit timers out of smaller 16-bit timers.
(Note: The Pico has a native 64-bit timer, but cascading is crucial for AVR, PIC, and STM32 chips).



Specialized Timer Types



General Purpose

The standard workhorses. Used for triggering delays, creating PWM signals, generating output waveforms, and capturing the timing of incoming digital signals.



Real-Time Clock (RTC)

Designed strictly to keep human time (Year, Month, Day, Hour, Min, Sec). Often runs on a separate, ultra-low-power domain with a coin cell battery backup.

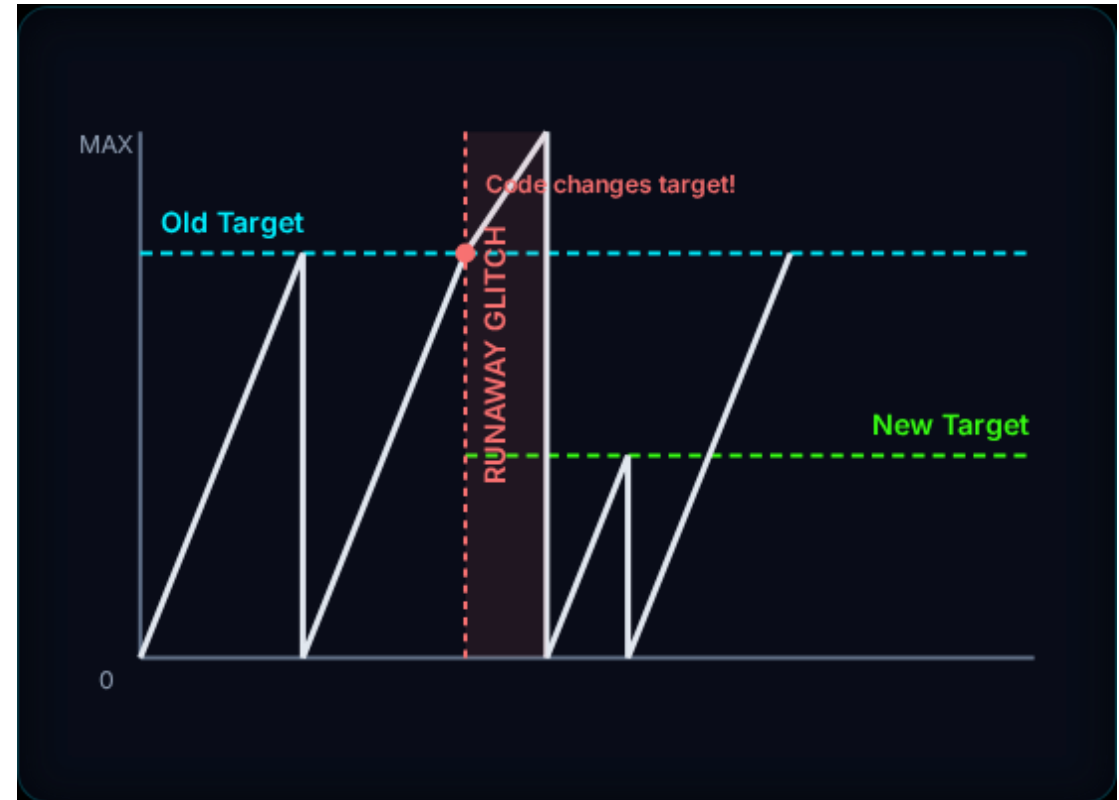


Watchdog Timer (WDT)

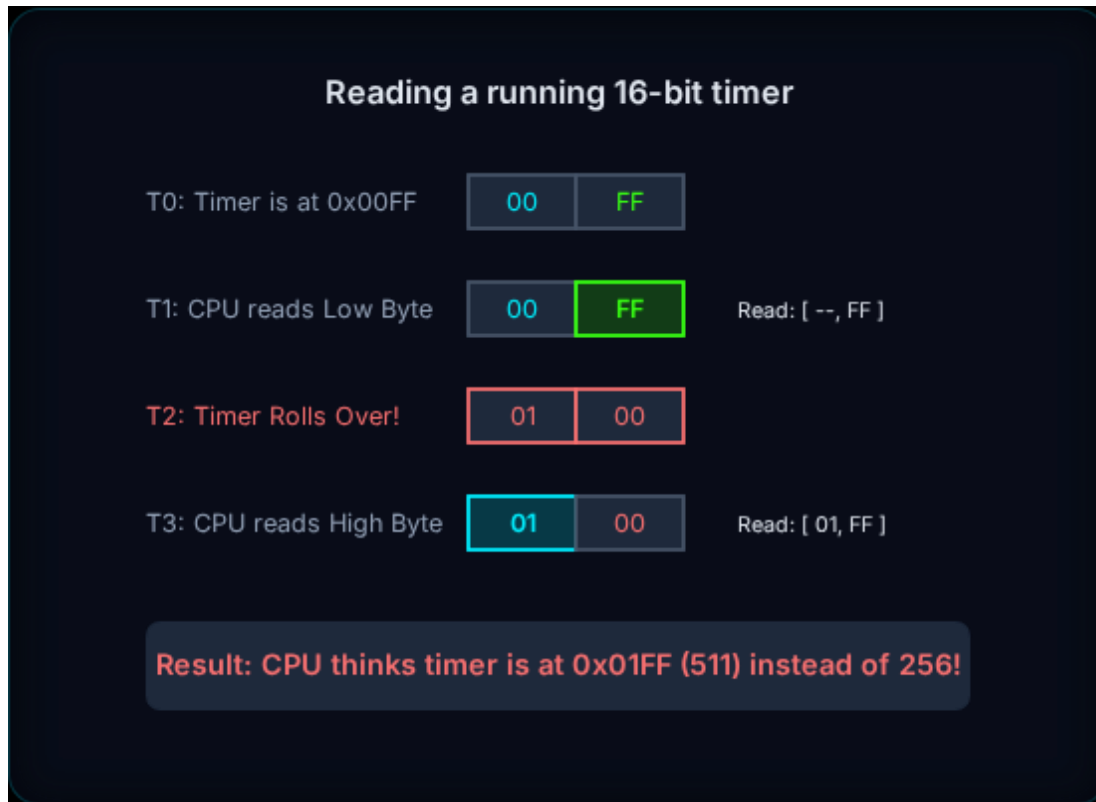
A safety failsafe. Software must continuously reset ("pet") this down-counting timer. If the code crashes and it hits zero, it forces a hard hardware reboot.

Dynamic Updates: The Runaway Timer

- ▶ **The Mistake:** Dynamically changing the Compare Match target while the timer is actively running without using Shadow Registers.
- ▶ **The Issue:** If the timer is already past the *new* target, it misses the match. It will blindly count all the way to its absolute maximum limit, overflow to zero, and *then* hit the new target.
- ▶ **The Impact:** This causes a massive, unexpected glitch in timing, a skipped beat in an RTOS, or a violent stutter in a PWM motor controller.
- ▶ **The Fix:** Always enable Preload/Shadow Registers. The hardware buffers your new value and applies it exactly at the rollover.



16-bit Reads on 8-bit Buses: Torn Reads



- ▶ **The Mistake:** Reading a multi-byte running timer directly. The CPU has to read the lower half, then the upper half sequentially.
- ▶ **The Issue:** If the timer rolls over perfectly in the middle of your two reads, you stitch together half of the old data and half of the new data.
- ▶ **The Fix:** Briefly disable global interrupts, or utilize the hardware's locking mechanism (many chips automatically freeze the upper byte shadow register the moment you read the lower byte) to ensure an **atomic read**.

Configuration Traps



Off-by-One (The N-1 Rule)

Hardware registers are **zero-indexed**. If you want a prescaler to divide by 100, and you write 100 to the register, it divides by 101 (counting 0 to 100 is 101 steps). Over time, this causes massive timing drift. Always use `Target - 1`.



The Infinite ISR

On bare-metal C, you must manually clear the timer's hardware interrupt flag at the end of the ISR. If you forget, the CPU exits the ISR, sees the flag is still up, and instantly re-enters, freezing the main program forever.



Clock Tree Illusions

Engineers often calculate timer periods assuming the timer runs at the main CPU speed (e.g., 100 MHz). However, peripheral buses often have their own hardware dividers, meaning Timer 2 might actually be running at 50 MHz.

What is Pulse Width Modulation (PWM)



Frequency

How often the pulse repeats.

$$f_{PWM} = 1 / T$$

Duty cycle

The share of each period spent HIGH.

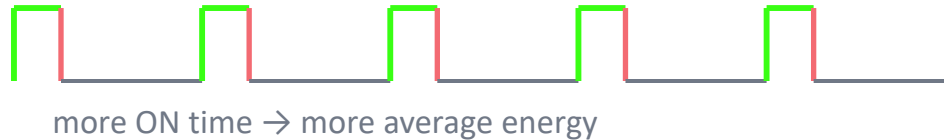
$$D = T_{on} / T \times 100\%$$

Frequency sets the rhythm; duty cycle sets the ON share.

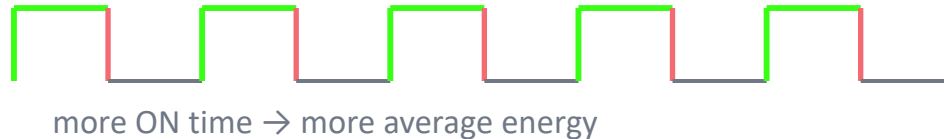
PWM = Fake Analog Voltage

Same voltage levels. Different ON time.

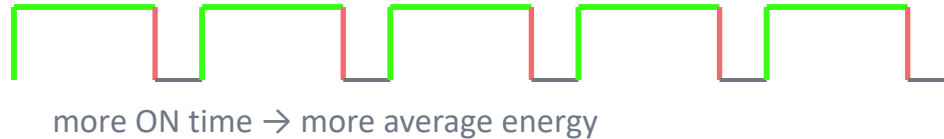
25%



50%



75%



The pin stays digital

- ▶ The output still switches between LOW and HIGH.
- ▶ A load, its inertia, or a filter averages the pulses over time.

Average drive $\approx D \times \text{supply level}$

Why use it?

Efficient switching avoids wasting the excess voltage as heat in a linear element.

LEDs

Motors

Power stages

Configuring PWM

Dedicated PWM peripheral

Built for richer waveform control

- ▶ Multiple synchronized channels and precise update timing
- ▶ Often adds complementary outputs and dead-time insertion
- ▶ May include fault / brake inputs for immediate shutdown
- ▶ Best fit: motor control and power-conversion stages

The exact peripheral name varies by MCU family.

General-purpose timer PWM

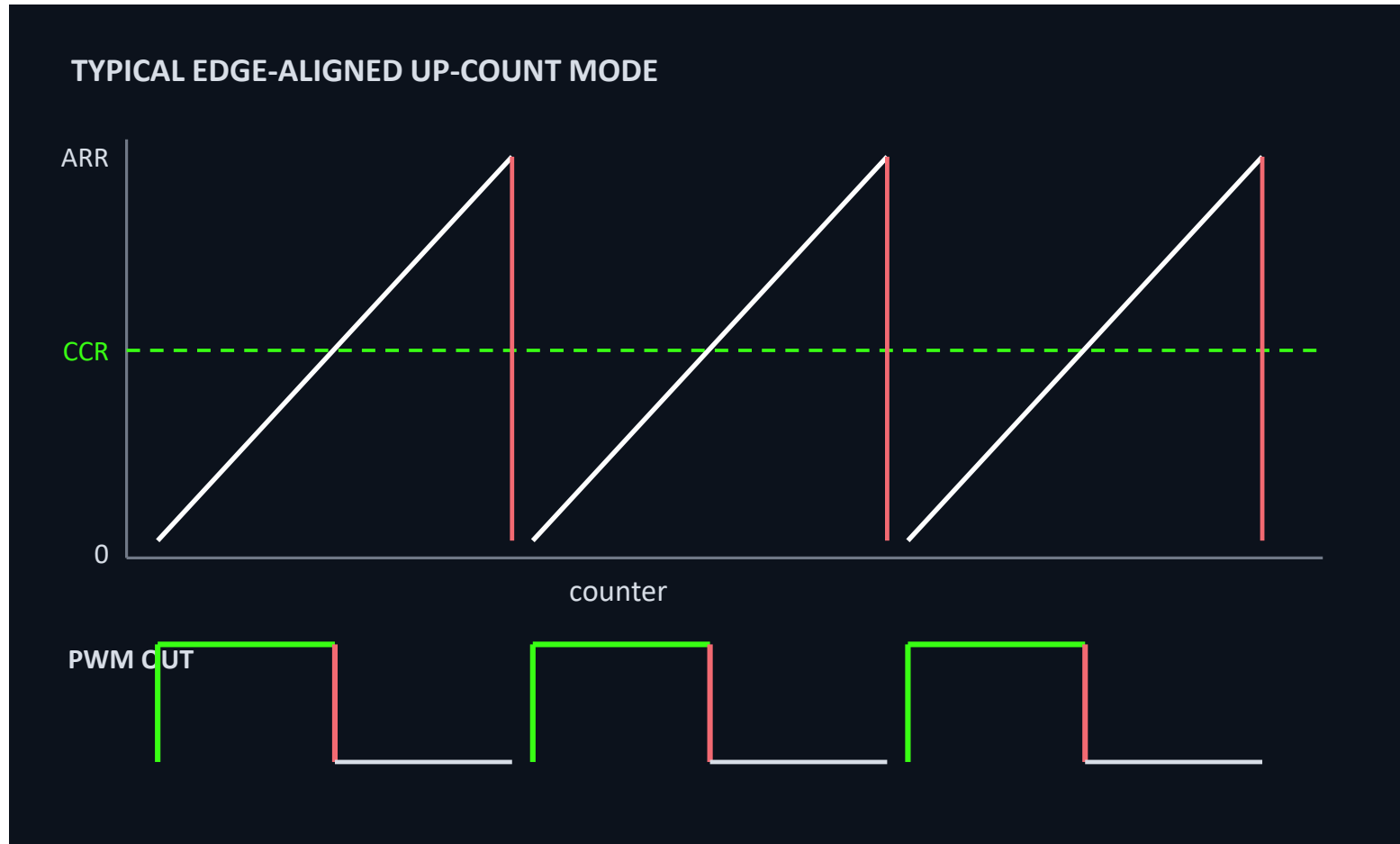
Uses timer output-compare behavior

- ▶ A counter defines position within each PWM period
- ▶ A period register defines when the counter rolls over
- ▶ A compare register defines when the output changes state
- ▶ Best fit: LEDs, buzzers, servos, and straightforward drives

Usually configured as one mode of an ordinary timer.

Same mechanism underneath: clocked counter + period + compare + output logic.

A timer becomes PWM when compare events drive the output pin



1

Count

Timer clock ticks advance the counter.

2

Reset

The period value ends one cycle and starts the next.

3

Compare

The compare match changes the output state.

Frequency: $f_{PWM} = f_{timer} / [(PSC+1)(ARR+1)]$

Duty: $D \approx CCR / (ARR+1)$

Exact formulas vary with counting mode and MCU.

The bridge: timer period sets PWM frequency; compare sets duty cycle.